```
function [retmap,retvisited,retsteps] =
astar_dynamic(mapfile,startlocation,targetlocation)

% ============================================================
% DYNAMIC A* / D*-LITE STYLE PATH PLANNER
%
% The algorithm:
%   1. Finds an initial path from start to target.
%   2. Moves the unit one cell at a time.
%   3. Re-reads the map after every movement.
%   4. Detects changes to the map.
%   5. Updates the path when obstacles change.
%
% Map convention:
%   0 = free space
%   non-zero = wall/obstacle
% ============================================================

% Load the initial map

map = map_convert(mapfile);

retmap = map;

[maprows,mapcols] = size(map);

% D*-Lite data structures

% g-value
g = inf(maprows,mapcols);

% rhs-value
rhs = inf(maprows,mapcols);

% The rhs value of the goal is zero
rhs(targetlocation(1),targetlocation(2)) = 0;


% km is used by D*-Lite when the start moves
km = 0;


% Previous position of the start
laststart = startlocation;


% Priority queue
openSet = [];

% Directions

directions = [
    -1  0       % up
     1  0       % down
     0 -1       % left
     0  1       % right
];

% Add the goal to the priority queue

key = calculateKey(targetlocation, startlocation, g, rhs, km);
```

```
61
62  openSet = [openSet; targetlocation key];
63
64  % Initial D*-Lite calculation
65
66  [g,rhs,openSet] = computeShortestPath(map, g, rhs, openSet, startlocation,
    targetlocation, km, directions);
67
68  % Variables for output
69
70  retvisited = [];
71  retsteps = [];
72
73  current = startlocation;
74
75  retvisited = [retvisited; current];
76  retsteps = [retsteps; current];
77
78  % Main dynamic loop
79
80  while ~(current(1) == targetlocation(1) && current(2) == targetlocation(2))
81
82      % Find the best neighbouring cell
83
84      neighbours = [];
85
86      for k = 1:4
87
88          neighbour = current + directions(k,:);
89
90          row = neighbour(1);
91          col = neighbour(2);
92
93          % Check map boundaries
94          if row >= 1 && row <= maprows && col >= 1 && col <= mapcols
95
96              % Check that it is not a wall
97              if map(row,col) == 0
98
99                  neighbours = [neighbours; neighbour];
100
101             end
102         end
103     end
104
105     % If there are no possible moves, stop
106
107     if isempty(neighbours)
108
109         disp('No path available.');
110
111         break;
112
113     end
114
115
116     % Choose neighbour with lowest g-value
117
118     bestNeighbour = [];
119     bestValue = inf;
120
```

```
121     for i = 1:size(neighbours,1)
122
123         row = neighbours(i,1);
124         col = neighbours(i,2);
125
126         if g(row,col) < bestValue
127
128             bestValue = g(row,col);
129
130             bestNeighbour = neighbours(i,:);
131
132         end
133     end
134
135
136     % Check whether a path exists
137
138     if isempty(bestNeighbour) || isinf(bestValue)
139
140         disp('No path available.');
141
142         break;
143
144     end
145
146
147     % Move the unit one cell
148
149     current = bestNeighbour;
150
151     retvisited = [retvisited; current];
152
153     retsteps = [retsteps; current];
154
155
156     % Start has moved
157     %
158     % D*-Lite updates km according to how far the start moved.
159
160     km = km + heuristic(laststart,current);
161
162     laststart = current;
163
164     % Check whether the map has changed
165
166     newmap = map_convert(mapfile);
167
168
169     if ~isequal(newmap,map)
170
171         disp('Map changed - updating path...');
172
173         % Find cells whose map values have changed
174
175         changed = find(map ~= newmap);
176
177         map = newmap;
178
179         retmap = map;
180
181         % Update affected vertices
```

```
182
183         for n = 1:length(changed)
184
185             [row,col] = ind2sub(size(map),changed(n));
186
187             changedCell = [row,col];
188
189
190             % Update the changed cell
191             [rhs,openSet] = updateVertex(changedCell, current, targetlocation, map, g,
    ...
192                 rhs, openSet, km, directions);
193
194             % Also update its neighbours
195
196             for k = 1:4
197
198                 neighbour = changedCell + directions(k,:);
199
200                 nr = neighbour(1);
201                 nc = neighbour(2);
202
203                 if nr >= 1 && nr <= maprows && nc >= 1 && nc <= mapcols
204
205                     [rhs,openSet] = updateVertex(neighbour, current, targetlocation,
    map, g, rhs, ...
206                         openSet, km, directions);
207                 end
208             end
209         end
210
211         % Recalculate only what is necessary
212
213         [g,rhs,openSet] = computeShortestPath(map, g, rhs, openSet, current,
    targetlocation, km, directions);
214
215     end
216
217 end
218
219
220 % Display result
221
222 plotmap(retmap,retsteps);
223
224
225 end
226
227
228 % HELPER FUNCTIONS
229
230
231 function key = calculateKey(position,start,g,rhs,km)
232
233 row = position(1);
234 col = position(2);
235
236 minValue = min(g(row,col),rhs(row,col));
237
238 key1 = minValue + heuristic(start,position) + km;
239
```

```
key2 = minValue;

key = [key1 key2];

end


function h = heuristic(a,b)

h = abs(a(1)-b(1)) + ...
    abs(a(2)-b(2));

end



function [g,rhs,openSet] = computeShortestPath( ...
    map,g,rhs,openSet,start,target,km,directions)


while true

    % Calculate start key

    startKey = calculateKey(start,start,g,rhs,km);


    % Find smallest key in queue

    if isempty(openSet)

        break;

    end


    bestIndex = 1;

    bestKey = openSet(1,3:4);


    for i = 2:size(openSet,1)

        currentKey = openSet(i,3:4);

        if currentKey(1) < bestKey(1) || (currentKey(1) == bestKey(1) && currentKey(2) <
bestKey(2))

            bestIndex = i;

            bestKey = currentKey;

        end
    end


    % Stop when start is consistent

    if ~isinf(g(start(1),start(2))) && ...
       g(start(1),start(2)) <= rhs(start(1),start(2))

```

```
300         if bestKey(1) >= startKey(1)
301
302             break;
303
304         end
305     end
306
307     % Remove smallest item
308
309     current = openSet(bestIndex,1:2);
310
311     oldKey = openSet(bestIndex,3:4);
312
313     openSet(bestIndex,:) = [];
314
315
316     currentKey = calculateKey( ...
317         current,start,g,rhs,km);
318
319     % Key is out of date
320
321     if oldKey(1) < currentKey(1) || (oldKey(1) == currentKey(1) && oldKey(2) <
    currentKey(2))
322
323         openSet = [openSet; current currentKey];
324
325         continue;
326
327     end
328
329     row = current(1);
330     col = current(2);
331
332     % If g > rhs
333
334     if g(row,col) > rhs(row,col)
335
336         g(row,col) = rhs(row,col);
337
338
339         % Update predecessors
340         for k = 1:4
341
342             predecessor = current + directions(k,:);
343
344             pr = predecessor(1);
345             pc = predecessor(2);
346
347             if pr >= 1 && pr <= size(map,1) && pc >= 1 && pc <= size(map,2)
348
349                 [rhs,openSet] = updateVertex(predecessor, start, target, map, g, rhs,
    openSet, km, directions);
350             end
351         end
352     else
353
354         % Set g to infinity
355
356         g(row,col) = inf;
357
358         % Update current
```

```
359         [rhs,openSet] = updateVertex(current, start, target, map, g, rhs, openSet, km,
    directions);
360 
361         % Update predecessors
362         for k = 1:4
363 
364             predecessor = current + directions(k,:);
365 
366             pr = predecessor(1);
367             pc = predecessor(2);
368 
369             if pr >= 1 && pr <= size(map,1) && pc >= 1 && pc <= size(map,2)
370 
371                 [rhs,openSet] = updateVertex(predecessor, start, target, map, g, rhs,
    openSet, km, directions);
372             end
373         end
374 
375     end
376 
377 end
378 
379 end
380 
381 function [rhs,openSet] =
    updateVertex(position,start,target,map,g,rhs,openSet,km,directions)
382 
383 
384 row = position(1);
385 col = position(2);
386 
387 % The target always has rhs = 0
388 
389 if row == target(1) && col == target(2)
390 
391     rhs(row,col) = 0;
392 
393 else
394 
395     % If this cell is a wall, it cannot be used
396 
397     if map(row,col) ~= 0
398 
399         rhs(row,col) = inf;
400 
401     else
402 
403         minValue = inf;
404 
405         % Look at neighbouring cells
406 
407         for k = 1:4
408 
409             neighbour = position + directions(k,:);
410 
411             nr = neighbour(1);
412             nc = neighbour(2);
413 
414             if nr >= 1 && nr <= size(map,1) && nc >= 1 && nc <= size(map,2)
415 
416                 if map(nr,nc) == 0
```

```
                    value = 1 + g(nr,nc);

                    if value < minValue

                        minValue = value;

                    end
                end
            end
        end


        rhs(row,col) = minValue;

    end
end

% Remove old entries for this cell from queue

if ~isempty(openSet)

    keep = ~(openSet(:,1) == row & openSet(:,2) == col);

    openSet = openSet(keep,:);

end

% If g != rhs, this cell is inconsistent and belongs
% in the priority queue

if g(row,col) ~= rhs(row,col)

    key = calculateKey(position, start, g, rhs, km);

    openSet = [openSet; position key];

end

end


function placestep(position,i)

% Plot a yellow rectangle and print the step number.

position = [16-position(1) position(2)];

position = [position(2)+0.1 position(1)+0.1];

rectangle('Position',[position,0.8,0.8], 'FaceColor','y');

c = sprintf('%d',i);

text(position(1)+0.2, position(2)+0.2, c, 'FontSize',10);

end
```